

Welcome Offer

Access to everything. Now 30% off. [Upgrade now](#)

React Data Routers Explained: Why React Router Changed Everything



Devendra Dhote

Follow

5 min read · 6 days ago



113



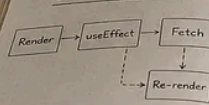
2



React Data Routers Explained: Why React Router Changed Everything

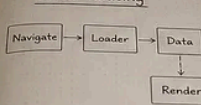
- ✓ Built-in Data Fetching
- ✓ Nested Loading & Error States
- ✓ Better UX. Less useEffect
- ✓ Smarter Routing for Modern Apps

Declarative Routing



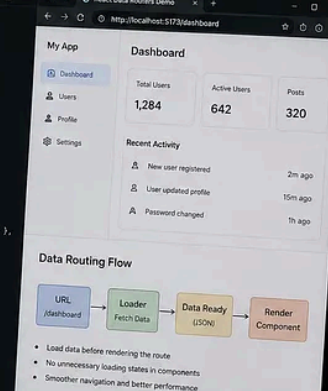
- Data fetched after render
- More loading states
- useEffect boilerplate
- Fetch on every navigation

Data Routing



- Data before render
- No unnecessary loading states
- Better UX
- Built-in data management

```
router.js
1 import {
2   createBrowserRouter,
3   RouterProvider,
4 } from "react-router-dom";
5
6 import App from "../App";
7 import Dashboard from "../routes/Dashboard";
8 import Users from "../routes/Users";
9 import Profile from "../routes/Profile";
10
11 const router = createBrowserRouter([
12   {
13     path: "/",
14     element: <App />,
15     children: [
16       { index: true, element: <Dashboard /> },
17       { path: "users", element: <Users />, loader: usersLoader },
18       { path: "profile", element: <Profile />, loader: profileLoader },
19     ],
20   },
21 ]);
22
23 export default function Root() {
24   return <RouterProvider router={router} />;
25 }
```



DATA ROUTING
= BETTER UX
LESS CODE

Clean Code
You Don't Know JS
Refactoring

If you've recently started learning React Router, you've probably noticed something confusing.

Some tutorials use this:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
  </Routes>
</BrowserRouter>
```

While others use this:

```
const router = createBrowserRouter([...]);
```

```
<RouterProvider router={router} />
```

At first glance, it feels like React Router simply introduced another syntax.

But that's **not** what happened.

React Router didn't introduce a new syntax.

It introduced a completely new **architecture**.

In this article, we'll understand why Data Routers exist, what problems they solve, and why almost every modern React application should prefer them.

• • •

A Quick Recap

In the previous article, we learned about Declarative Routing.

Our routes looked something like this:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/dashboard" element={<Dashboard />} />
  </Routes>
</BrowserRouter>
```

This worked perfectly.

React Router looked at the current URL and decided which component should be rendered.

Simple.

But as applications became larger, developers started noticing some problems.

• • •

The Biggest Problem

Imagine a dashboard.

When the user visits:

`/dashboard`

What usually happens?

Navigate

↓

Dashboard Component Renders

↓

useEffect Runs

↓

API Call

↓

Loading Spinner

↓

Response Arrives

↓

Component Re-renders

The page renders before the data exists.

That's why many applications briefly show loading spinners or skeleton screens before the actual content appears.

There's nothing wrong with this approach.

In fact, we've all written code like this.

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

But think about it for a moment.

The route already knows that the Dashboard needs data.

So why should the component wait until after rendering to start fetching it?

That question eventually led to Data Routers.

. . .

React Router's New Philosophy

The React Router team introduced a new idea.

A route shouldn't only know what to render. It should also know how to load its data, handle form submissions, recover from errors, and manage navigation.

In other words,

a route becomes a complete unit.

Instead of only describing UI,

it describes the entire lifecycle of that page.

. . .

Meet Data Routers

Instead of writing routes using JSX,

we create them as JavaScript objects.

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <Home />
  }
]);
```

And instead of rendering `BrowserRouter`, we render:

```
<RouterProvider router={router} />
```

This small change unlocks an entirely new set of capabilities.

. . .

Why Route Objects?

You may wonder...

Why did React Router stop using JSX?

Because a route now contains much more information than just a component.

A modern route can define:

- Which UI to render
- How to fetch data
- How to submit forms
- How to handle errors
- How to lazy load code
- How to revalidate stale data

Trying to express all of this inside `<Route>` components would quickly become difficult.

Objects are much better suited for configuration.

. . .

Understanding Loader

Imagine opening a users page.

Without Data Routers:

Render

↓

useEffect

↓

Fetch Users

↓

Render Again

With Data Routers:

Navigate

↓

Loader Runs

↓

Data Arrives

↓

Component Renders

Notice the difference.

The data is loaded **before** the component is rendered.

This creates a much smoother experience.

A loader looks like this:

```
{
  path: "/users",
  loader: async () => {
    return fetch("/api/users");
  },
  element: <Users />
}
```

Inside the component:

```
const users = useLoaderData();
```

No `useEffect`.

No local loading state.

No extra fetch logic.

The route already handled it.

• • •

Why This Is Better

Think about a movie streaming platform.

When a user opens a movie page, the application already knows which movie information is required.



Instead of rendering an empty page and fetching later,

the router can fetch everything first and then render the completed screen.

The routing system becomes smarter because it understands data dependencies.

. . .

Built-in Form Handling

Traditional React applications usually handle forms manually.

User Clicks Submit



```
preventDefault()
```

↓

```
fetch()
```

↓

```
Handle Errors
```

↓

```
Navigate
```

Data Routers introduce another concept:

```
action() .
```

Instead of manually wiring everything together,

the router handles form submissions.

```
{
  path: "/login",
  action: async ({ request }) => {
    // Process Form
  }
}
```

This makes forms feel much closer to how browsers naturally work.

. . .

Better Error Handling

Consider this situation.

A user visits:

```
/products/9999
```

The product doesn't exist.

Or maybe an API request fails.

Traditionally, we write lots of `try...catch` blocks.

Data Routers simplify this using `errorElement`.

```
{
  path: "/products/:id",
  element: <Product />,
  loader: productLoader,
  errorElement: <ProductError />
}
```

Whenever something goes wrong,

React Router automatically renders the error UI.

The same mechanism handles:

- Missing routes
- Loader errors

- Action errors
- Rendering errors

One place.

One solution.

• • •

Revalidation

Imagine a todo application.

You create a new todo.

Normally you would write another fetch request to update the UI.

Data Routers can automatically re-run loaders after actions.

This process is called **Revalidation**.

Your UI stays synchronized without manually writing additional fetching logic.

• • •

Lazy Route Loading

Large applications may contain hundreds of pages.

Loading everything on the first visit isn't efficient.

Data Routers support lazy loading directly at the route level.

Instead of downloading the entire application,

only the required routes are loaded.

Smaller bundle.

Faster startup.

Better user experience.

. . .

Declarative vs Data Routing

Let's compare both approaches.

Declarative Routing	Data Routing	Uses <code>BrowserRouter</code>	Uses <code>createBrowserRouter()</code>
JSX-based routes	Object-based routes	Focuses mainly on navigation	Focuses on navigation and data
		Data fetching usually happens in <code>useEffect</code>	Data fetching happens inside <code>loader()</code>
Manual form handling	Built-in <code>action()</code> support	Manual error handling	<code>errorElement</code>
Good for simple applications	Ideal for modern applications		

Neither approach is “wrong.”

Data Routers simply solve more real-world problems.

. . .

Which One Do Companies Prefer?

Today, most production React applications are data-driven.

Every page usually needs:

- Authentication
- API Requests
- Forms
- Error Recovery
- Permissions
- Optimistic Updates
- Fresh Data

Because of this, teams increasingly prefer the Data Router architecture for **new React Router projects**.

That doesn't mean companies using Declarative Routing are outdated.

Many large codebases were built before Data Routers existed and continue to work perfectly.

Migration takes time, especially for enterprise applications with hundreds of routes.

So in practice:

- Existing projects often continue using Declarative Routing.
- New applications commonly adopt Data Routers from the beginning.

The choice usually depends on the age of the project, not on developer skill.

. . .

Does This Mean `useEffect` Is Dead?

Absolutely not.

This is one of the biggest misconceptions.

`loader()` is excellent for **route-level data**.

`useEffect` is still the right choice for things like:

- WebSocket connections
- Timers
- Analytics
- Browser APIs
- Event listeners
- Synchronizing with external systems

Both have different responsibilities.

. . .

Mental Model

The easiest way to remember the difference is this:

Declarative Routing answers one question:

| Which component should I render?

Data Routing answers four questions:

- Which component should I render?
- What data should I load?
- How should forms be handled?
- What should happen if something goes wrong?

That's why Data Routers feel like a natural evolution rather than a replacement.

. . .

Final Thoughts

React Router didn't replace Declarative Routing because it was bad.

It evolved because modern web applications evolved.

Today's applications aren't just collections of pages.

They're full of authenticated dashboards, real-time updates, server communication, forms, permissions, and complex user interactions.

Data Routers acknowledge this reality.

By moving data loading, form handling, and error management into the routing layer, React Router gives us an architecture that's easier to scale, easier to maintain, and much closer to how modern applications are actually built.

In the next article, we'll go one step deeper and explore how React Router works under the hood — how routes are matched, how nested routes are resolved, how navigation works internally, and what really happens when you change the URL.

Happy Coding ❤️ .

Software Development ✓

Programming ✓

JavaScript ✓

Routing +

Web Development ✓



Written by Devendra Dhote

343 followers · 8 following

Follow

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.